# Parte 2 - MQTT e visualização no Node-RED

Guilherme Yamada Dantas - rm568506 | CardioIA Conectada | 30/10/2026

## Arquitetura implementada

O firmware coleta temperatura e umidade do DHT22, aceleração do MPU6050 e BPM sintético controlado por potenciômetro. Não há medição cardíaca real. A cada cinco segundos uma amostra entra na fila RAM limitada do ESP32. A tarefa de rede publica JSON em cardioia/rm568506/telemetry, no broker broker.hivemq.com, porta 1883. O Node-RED assina esse tópico, valida os campos e alimenta o FlowFuse Dashboard 2.0. O Node-RED pode funcionar como camada Fog no computador local; o broker remoto representa a camada de comunicação em nuvem. O dashboard não foi hospedado publicamente.

O contrato usa synthetic=true, device_id, session_id, seq, uptime_ms, age_ms, temperature, humidity, bpm e acceleration. O firmware acrescenta device_id, sensor_error, queued e dropped. Temperatura e umidade inválidas são null; não são convertidas em zero. A origem temporal é o uptime do ESP32, sem relógio UTC sincronizado. A visualização usa explicitamente o horário de recepção: amostras de backlog não devem ser interpretadas como aquisição contemporânea. Isso evita inventar timestamps clínicos precisos.

## Comunicação e qualidade dos dados

A biblioteca PubSubClient publica em QoS 0. A remoção da fila acontece quando publish retorna sucesso, o que não constitui confirmação de consumo pelo Node-RED ou armazenamento no broker. Pode haver perda em falhas de transporte e duplicação após reconexão. O dashboard deduplica até 2000 identificadores device_id:session_id:seq em memória; reiniciar o serviço limpa esse histórico. Não existe garantia fim a fim de exactly-once nem retenção clínica.

Mensagens de sequência regressiva no mesmo dispositivo/sessão são rejeitadas. age_ms registra a idade no momento do envio; acima de 15 segundos, o texto marca BACKLOG e não representa estado atual. A entrada rejeita JSON inválido, synthetic diferente de true, tipos incorretos (incluindo aceleração, uptime e idade), falha de sensor, sequência inválida e grandezas fora das faixas didáticas. Os intervalos admitidos são 0-250 BPM, -40-80 °C e 0-100% de umidade. A rejeição preserva o último valor válido, mas o indicador passa a SEM DADOS RECENTES quando a última recepção válida tem mais de 15 segundos, com atualização a cada cinco segundos. O gráfico e o gauge podem manter valores antigos; o estado avisa que eles não representam telemetria atual.

O gráfico representa BPM e mantém até 720 pontos/uma hora. O medidor apresenta temperatura ambiente. O texto indica alerta para BPM >120 ou temperatura >38 °C; exatamente 120 e 38 permanecem dentro das regras. As regras são educacionais e não diagnosticam taquicardia ou febre. O MPU6050 comprova uma segunda modalidade de sensoriamento; aceleração não é transformada em diagnóstico de imobilidade.

## Configuração reproduzível

Na raiz do projeto, npm ci instala as versões fixadas em package-lock.json. Executar npm run broker cria um broker de teste em 127.0.0.1:18883. Em outro terminal, MQTT_HOST=127.0.0.1 MQTT_PORT=18883 npm run dashboard inicia Node-RED em 127.0.0.1:1887. O dashboard fica em /dashboard/cardioia e o editor em /. O arquivo dashboard/flows.json pode ser importado em outra instalação com @flowfuse/node-red-dashboard instalado. As variáveis MQTT_HOST e MQTT_PORT devem ser definidas antes da inicialização.

Para usar o broker do firmware, definir MQTT_HOST=broker.hivemq.com e MQTT_PORT=1883. A publicação de teste é executada por npm run test:mqtt, com MQTT_URL correspondente ao broker. Esse publisher roda no computador e se identifica como host-qa. Ele não é um ESP32 emulado e seu teste não comprova execução do Wokwi. A configuração do serviço restringe HTTP ao loopback e não altera instalações globais de Node-RED.

## Testes e evidências

O teste de integração usa um cliente MQTT real, sem substituir o protocolo por funções locais. Ele publica uma série de BPM, verifica chegada no endpoint de estado do fluxo, testa os dois limiares, mensagem duplicada, marcador synthetic=false, tipo incorreto, leitura null, expiração e recuperação. O arquivo evidence/mqtt-local.json registra o resultado e os snapshots. A imagem evidence/dashboard-local.png mostra o gráfico, medidor de 39,2 °C e texto de alerta reais em execução com publisher host.

O teste equivalente com broker público passou e está registrado separadamente em evidence/mqtt-public.json. A presença de resultados locais não comprova nuvem; somente o registro do broker remoto pode ser usado para essa afirmação. Logs de inicialização identificam versão, carregamento de widgets e conexão. A execução ESP32/Wokwi também foi realizada separadamente: evidence/wokwi-final-mqtt.jsonl registra o recebimento e dashboard-wokwi-final.png mostra o alerta após BPM sintético de 180. Consulte o relatório Edge e a revisão final.

## Segurança, limites e evolução

O broker público não tem autenticação, isolamento de tópico ou TLS nesta configuração. Qualquer pessoa pode observar ou injetar mensagens, e synthetic=true não autentica ninguém. Não enviar dados pessoais. Um cenário real exigiria broker privado, TLS com validação de certificados, ACL por dispositivo, credenciais protegidas, persistência, retenção, trilha de auditoria, confirmação de entrega e validação clínica. O protótipo não demonstra esses controles.

O npm audit foi executado e preservado em evidence/npm-audit.json; há avisos transitivos que o ajuste automático compatível não resolveu. Por isso o editor permanece somente em loopback, e não se declara prontidão para produção. Referências técnicas consultadas: nodered.org/docs/getting-started/local e dashboard.flowfuse.com/getting-started.html. Referências acadêmicas: FIAP, Fase 3, Cap08 (Cloud) e Cap09 (Edge/Fog). Grafana é opcional e não foi adotado.